

OAuth 2.0 and OpenID

Introduction

This objective of this assignment is to learn how authentication and authorization are handled in real world. Rather than managing user credentials directly, the task is delegated to trusted third-party such as Microsoft EntraID. By using Microsoft EntraID, a user can use their credentials to gain a token so that they gain access to protected resources. The main task of this assignment to further implement a flask application to include an authentication mechanism that relies on Microsoft EntraID for authentication and authorization. This report will go through the background theories, the details of design and an evaluation of the given solution.

Technical Background

Credential-based authentication

Credential-based authentication is a method where a user has to provide credentials, typically username and password, in order to verify the authenticity of the user. This assignment delegates authentication to a trusted third party, Microsoft EntraID, rather comparing the stored credentials with the provided credentials like the traditional way.

OAuth 2.0 and OpenID Connect

OAuth 2.0 is authorization protocol that allows the client to work on protected resources without having to give credentials repeatedly [3]. The client starts by requesting authorization from the resource owner. Once the authorization is granted, an authorization code is given to the client and then client can send the code to the authorization server. If the code is valid, the authorization server returns an access token. The access token can be used to perform tasks on the protected resources as long as the token remains valid.

OpenID connect is an authentication layer built on top of OAuth 2.0 protocol to add extra security. This layer forces the user to provide their credentials so the authorization server can verify the authenticity of the user. [4]

For this assignment, the authorization server, Microsoft EntraID, is used for authentication. Upon successful authentication, the web-application receives an access token.

TLS certification and Certificate Authority [5]

Transport layer security's job is to encrypt communications between a client and a server to prevent tampering or eavesdropping by attacker. Certificate Authority, a trusted third party that certifies the trustworthiness of websites, is used to create a certificate for the desired website and issues it to the web server. To obtain a certificate from the CA, the web site owner has to submit a formal request to the CA for review. The certificate is what the web server sent back to the client to verify the webserver's trustworthiness.

Single-page-application (SPA)

Single-page-application is one way to structure a web application. This method allows the front end to have data handling logics, a page navigation system that does not fully reload pages [1] and client-side rendering (browser renders the UI by using integrated JavaScript/Typescript). Because the front-end system is the one that handles data, the data have to come from a back-end system. Thus, this architecture is depended on APIs for communication between the back-end system and front-end system. [1] Additionally, some data are stored in the browser's cache because the front-end system is able to handle data.

Server-side-render (SSR)

Server-side-render is the opposite SPA, and it is the traditional way of rendering a website. Web applications architecture that utilises this render method must send a new request from the front end to the back end every time a user click on a new link. This allows the server to send back already-rendered HTML, allowing users to see the content of the requested web page immediately while the Javascript part is being downloaded [1].

Design and Implementation

Architecture

The application follows a server-side rendered architecture using Flask framework. The authentication logic and API calling happens on the server, while the browser fully renders the given HTML pages. Unlike single-page-application, which uses client-side-rendering, flask uses server-side-rendering. That means that the token does not leave the server, and it is not stored in the browser's cache. This keeps the token safe from any attacks on the browser. Additionally, flask is the most reasonable choice for this assignment because of low the requirements. Flask is minimalist and easy to learn [2]. This further reenforce the argument.

Authentication flow

The flow of the solution can flow two ways depended on the status of the current user. When an unauthenticated user attempts to log in, the “/login” route calls “auth.log_in()” from the “identity.web” library, which redirects the user to Microsoft’s login page. If the unauthenticated user tries to access the other functionalities (e.g edit profile), the website will call “logout()” and returns to the home page by calling “index()”. To verify the user, the user has to first fill in their Microsoft credentials. Upon clicking on enter, EntraID redirects back to the callback URI “/getAToken”, where “auth.complete_log_int” exchanges the authorization code for a token. If an error occurred during this process, an error page is shown; Otherwise the user is redirected to the home page.

Logout is handled by “logout()”, which use “auth.log_out()”. The fuction clears the session and redirects the user to Microsoft’s logout page.

Identifying authenticated users

After login, the authenticated user’s profile is retrieved using “auth.get_user()”, which reads the identity of the user in the current session. The username is then used to display on home page. For API calls, “auth.get_token_for_user(SCOPES)” is used to obtain a valid access token. If the user is unauthenticated or the session has expired, the website redirects the user back to the login flow.

Handling unauthorized access

Routes like “/profile” and “/users” are protected. Every time an unauthenticated user try access them, “auth.get_token_for_user()” is called to get the token for the current session. If no token is found, the user is logged out and redirected to the home page.

Graph API integration

Two APIs are used in the solution, GET and POST. The “/profile” route can either do a GET request to get the user’s profile after signing in, or a PATCH request that updates the fields submitted via an HTML form. Furthermore, the “/user” route can also send a GET request to retrieve a list of all users in the tenant. All requests include the access token to ensure that the

requests came from an authenticated user. The web application sends requests to the provided graph URL, which is where the profiles are stored.

OpenID Scopes

The OpenID Scopes that are used in the application are User.Read, User.ReadWrite, User.ReadWrite.All. The first one allows the application to read the signed-in user's profile via API calls. The second one allows the signed-in user to update their own profile. Finally, the last one allows reading and updating other user's profiles in the tenant.

TLS Certificate

This application is deployed on a virtual machine with the domain name "gng000.inf2310.net", a TLS certificate is needed to ensure that the virtual machine's public key is trustworthy. TLS certificate was obtained by using Let's Encrypt via Certbot, which enabled HTTPS for the domain, which in turn, encrypts the authentication flow.

Evaluation

When a user visits the website without logged in, they are presented with a welcome message with a login link. If the user tries to access "show_users" or "edit_profile", it will redirects the user back the home page.

Login to gain access to main web page

Menu

- [login](#)
- [Show Users](#)
- [Edit Profile](#)
- [Logout](#)

Clicking on the login functionality, it will redirects the user to Microsoft's login page, where they can use credentials to login. Upon successful login, the website is redirected to

“/getAToken”, where authorization code is exchanged for an access token. The website then returns to home page, displaying the username in the message.

Welcome Giang

Menu

- [login](#)
- [Show Users](#)
- [Edit Profile](#)
- [Logout](#)

If logged in, the user will have access to “show_users” and “edit_profile”.

Tenant Users

- Abbas
- Alham Ghafoor
- alrod7901
- analb8514
- ancarmay
- Anders
- aneri9852
- aureo0625
- Aymeric
- bam012
- cabar9818
- Dario
- daski7154
- Dina Benaddou
- dugam1065
- Eskil Bjørnsbakk Heines
- Einar Jakobsen Holthe
- Elmi Hashi
- erda4096
- esk0091
- fam003
- Gabriel
- Giang
- haras8970
- Håvard Dagnborg
- Håvard Dagnborg
- User420421
- Jens Sonaby
- kensd1747
- kinvor0161
- krkul1675
- lbe110
- lifax5536
- likli8138
- Magnus H.
- mbr188
- mskd039
- mwi101
- Markus Leander Wilhelmssen
- nache7383
- niscoe4972
- Oystein
- oscon0619
- Oskar Sundberg
- pelan8244
- peno020
- Rasmus
- rydzec2130

Update Your Profile

User ID
92c4a146-061-4012-b024-8f
Display Name
Giang
Given Name
Giang
Surname
None

[back](#)

Update Your Profile

Profile stored!

User ID
92c4a146-061-4012-b024-8f
Display Name
Giang Nguyen
Given Name
Giang
Surname
None

[back](#)

Tenant Users

- Abbas
- Ahsan Ghafoor
- alrod7901
- analb8514
- ancarmay
- Anders
- aneri9852
- anree0625
- Aymeric
- bam012
- cabar9818
- Dario
- dasli7134
- Dina Benaddou
- dugam1065
- Eskil Bjørnbakk Heines
- Einar Jakobsen Holsbø
- Elmi Hashi
- erdal4096
- esk091
- fam003
- Gabriel
- Giang Nguyen

Conclusion

This assignment required implement and deploying a flask application on a remote virtual machine. The application delegates authentication and authorization to a third party, Microsoft EntraID, which allows users to do Microsoft Graph API calls to retrieve other user's profiles and updating the user's profile as well. Overall, the solution did fulfils the requirements of this assignment.

References

- 1) *Vue.js*. (n.d.). <https://vuejs.org/guide/extras/ways-of-using-vue.html#single-page-application-spa>
- 2) *Which is the best Python web framework: Django, Flask, or FastAPI? | The PyCharm blog*. (n.d.). The JetBrains Blog. <https://blog.jetbrains.com/pycharm/2025/02/django-flask-fastapi/>
- 3) *RFC 6749: The OAuth 2.0 Authorization Framework*. (n.d.). IETF Datatracker. <https://datatracker.ietf.org/doc/html/rfc6749>
- 4) OpenID Foundation. (2023, July 20). *How OpenID Connect works - OpenID Foundation*. OpenID Foundation - Helping People Assert Their Identity Wherever They Choose. <https://openid.net/developers/how-connect-works/>
- 5) Goodrich, M. A., & Tamassia, R. (2010). *Introduction to computer security*. Page 334 - 336